

表 2-1 主要符号（续）

符号	定义说明	取值范围或量纲
L, U	在相同图输入、核心数与硬件配置下，对所有合法方案均成立的理论下界与已有合法方案的时间上界	cycle
$L(P)$	仅在指定候选方案 P 的特定结构约束下成立的时间下界（不作为全局下界）	cycle
U/L	当前最佳方案的 Makespan 与理论下界之比，用于评估当前方案逼近理论极限的程度	无量纲比值
T_{solve}	求解算法从读取输入到输出合法方案文件的端到端墙钟运行时间	s（秒）

在官方输入文件中，非核外搬运计算算子 $v \in V$ 的 **cycles** 字段可能为 0。根据官方评测程序逻辑，计算单元在单核心执行推进时确保每个计算算子至少占用 1 个执行时钟周期，因此在建立数学分析模型与计算计算负载下界时，算子执行周期定义为 $d_v = \max(1, \text{cycles}_v)$ （单位为 cycle）。需要特别说明的是， d_v 仅适用于具备固定计算时长的算子集合 V ；对于核外数据搬运操作（COPY_IN 与 COPY_OUT），其执行耗时由张量大小 b_t 与实时动态共享带宽池决定，不适用该静态公式。

2.4 评价指标与计算方法

2.4.1 完成时间与多核加速比

多核心调度方案的质量首要由 Makespan $M_q(P)$ 衡量。为了评估多核并行化相对于单核执行的性能提升程度，根据题面第 5 页脚注 2 的明确规定，多核心调度方案的平均加速比（Speedup）采用逐用例比值的算术平均方法进行计算：

$$\bar{S}_{q,K} = \frac{1}{100} \sum_{i=1}^{100} \frac{B_i}{M_q(P_{i,K})}, \quad (2-1)$$

其中 B_i 为官方评测系统针对第 i 张图在单核心条件下的基线 Makespan， $P_{i,K}$ 为算法针对第 i 张图在 K 个核心下生成的调度方案。必须说明的是，平均加速比是逐图加速比值的算术平均值，而非全部用例单核时间总和除以多核时间总和。题面官方基线 B_i 由通用的单核启发式调度算法给出，并不保证是单核条件下的数学全局最优解，因此在部分用例中可能出现加速比略大于核心数 K 的情况。

对于问题三，题面要求评估引入只读 Cache 带来的相对性能增益。评测时保持问题三所生成的调度方案 P 完全不变，分别在启用 Cache 和关闭 Cache（即回退至问题二纯 DDR 模式）两种配置下运行评测程序，分别记录其 Makespan 为 $M_3(P)$ 和 $M_2(P)$ 。单用例的

Cache 加速比定义为 $M_2(P)/M_3(P)$ ，多用例汇总时同样报告这 100 对结果比值的算术平均值。

2.4.2 额外 DDR 搬运量与缓存命中率

次要方案质量指标包括数据搬运量与缓存命中率：

- **总额外 DDR 数据搬运量 $D_q(P)$ ：**官方评测程序统计调度后执行的所有 COPY_IN 和 COPY_OUT 搬运字节总数，减去原始计算图固有输入输出张量所需的基准搬运字节数。在问题三中，官方评测系统保持相同的统计口径，并不在 Cache 命中时将该统计字段直接扣减，因此该字段反映的是切分引入的总通信需求，而非物理 DDR 总线的实际净流量；
- **Cache 字节命中率 H ：**定义为命中 Cache 的数据总字节数占有所有发起 Cache 查询的总数据字节数的比例，即

$$H = \frac{\text{hit_bytes}}{\text{hit_bytes} + \text{miss_bytes}}. \quad (2-2)$$

当用例中不存在任何可查询的 COPY_IN 操作时，官方程序将分母为零的情形约定为 $H = 0$ 。该指标严格按字节量统计，而非按访问请求的发生次数统计。

2.4.3 端到端求解耗时

算法的求解效率通过端到端求解时间 T_{solve} 评估，度量从程序启动、解析输入图 JSON，到完成拓扑分析、方案构造、内部筛选及输出方案文件全过程的实际运行墙钟耗时（Wall-clock Time）。

原题第 5 页脚注 1 指出：“此处‘高效地’代表着算法应该避免使用暴力迭代搜索机制，以单个用例能在 5~10 分钟内给出结果为佳。”我们对该项要求的理解为：

- 该项表述属于针对求解算法工程效率与设计理念的指导性建议，强调算法应当利用计算图的固有结构与硬件特性进行针对性构造，避免盲目的大规模蒙特卡洛枚举或无剪枝的暴力迭代；
- 5~10 分钟是算法在常规计算硬件上的用例级运行耗时建议，并非超过 600 秒即被判定无效的硬性淘汰线，亦非要求算法必须耗满 5 分钟方可输出。本队算法在保证方案高质量的前提下，追求极高的求解速度，单用例求解耗时通常控制在数秒至数十秒之间。

2.4.4 评测方法与调用预算

在算法求解与评测交付中，明确区分在线候选评分（作为求解算法的一部分用于分支决策，计入求解墙钟时间）与外部官方复评（求解完成后独立运行以验证方案合法性并输出最终官方主成绩）：

- **E0（官方离散事件评测程序）：**官方提供的未修改基准评测脚本（code/multicore_cut_evaluate_problem_*.py）。在问题三求解过程中，算法基于归约树排序理论生成有限个候选方案后，直接调用 E0 进行在线完整打分验证（单用例调用预算上限为 3 次）；而在外部复评阶段，本文汇报的所有主实验成绩、最终 Makespan 以及附录逐用例结果，均以 E0 的独立运行输出为最终确认标准；
- **E1（批量精确评估工具）：**队内依据官方规则实现的轻量级独立评估实现。在问题一求解器中，用于对有限个候选方案（包括基础划分、局部调整及分支重分配方案）进行在线模拟打分与选优，单用例在线调用次数严格限制在至多 10 次以内；

- **E2（场景二轻量精确评估工具）**：队内针对问题二场景（同核子图合并为单一 Task、跨核以 COPY 依赖推进）实现的快速评估工具（在固定源码 P2 程序模块中由 `SceneBEvaluator` 提供支持）。在问题二求解器（`adaptive_hypergap_guarded.py`）中，算法基于超图多阶段划分与空隙插入构造出 `baseline`、`gap` 与 `hypergap` 等有限候选后，调用在线评分适配器进行打分（单用例在线调用上限为 3 次），并根据所返回的 `Makespan` 与额外数据搬运量自适应选取最佳方案；若 E2 接口调用异常则自动回退至 E0。

所有求解器内部的在线打分调用及其等待时间均严格计入算法的端到端求解时间 T_{solve} 中，未在求解阶段隐瞒在线评价开销。

3 多核调度的数学模型与求解框架

求解器输入为计算图 G 、核心数 K 与硬件配置。合法输出方案 $P = (\pi, a, \sigma)$ 精确映射至官方评测约定的两个提交字段：算子所属子图字典 `node_to_subgraph`，以及各核心子图执行全序 `core_schedules`。三问求解均以最小化离散事件仿真完成时间（`Makespan`）为首要目标，兼顾数据搬运代价。

3.1 多核切分与调度模型

3.1.1 决策变量与方案表示

多核心调度算法的核心任务是在满足数据流依赖与硬件物理约束的前提下，共同确定算子到子图的分组、子图到计算核心的指派以及各核心内部子图任务的执行先后时序。

设 $V \subset O$ 为原始计算图中除跨边界核外搬运操作 `COPY_IN` 与 `COPY_OUT` 之外的待调度操作集合（此集合包含全部计算算子以及子图内部的数据搬运操作）。方案的三个决策维度形式化定义如下：

1. **算子划分映射 π** ： $\pi : V \rightarrow \mathcal{S}$ 为每个算子 $v \in V$ 分配一个子图编号，所有非空子图组成的集合记为 $\mathcal{S}$ ；
2. **核心分配映射 a** ： $a : \mathcal{S} \rightarrow \{0, 1, \dots, K-1\}$ 为每个子图 $s \in \mathcal{S}$ 指定负责处理该子图计算任务的核心编号；
3. **核心内调度序列 σ_k** ：对于每个核心 $k \in \{0, 1, \dots, K-1\}$ ， σ_k 为指派给该核心的子图子集 $\{s \in \mathcal{S} \mid a(s) = k\}$ 给出一个严格的全序执行排列。

三项决策共同构成了完整的多核心切分与调度方案，记为三元组 $P = (\pi, a, \sigma)$ 。该数学表示与官方提交契约（`contest_io.py`）中的持久化结构完全对应：`node_to_subgraph` 字段直接对应映射 π ，`core_schedules` 字段直接对应核心执行序列集合 $\{\sigma_k\}_{k=0}^{K-1}$ 。单有算子分组而不指定执行核心，或者指定了分配而不确定同一核心上的执行次序，均无法在底层评测系统中推进离散事件时钟。

3.1.2 数据依赖与执行顺序约束

为了严谨刻画多核心场景下由数据依赖与核心内排队顺序交织构成的时序先后关系，我们建立子图层级的扩展有向拓扑：

- **子图间数据依赖边集 $E_{\mathcal{S}}$** ：若子图 s 内算子与子图 t 内算子之间存在直接操作依赖（`opto-op`）、跨子图张量生产—消费关系，或经由官方派生逻辑生成的完整跨边界数据搬运路径（ $s \neq t$ ），则在子图之间引入一条有向依赖边 $(s, t) \in E_{\mathcal{S}}$ ；

- **核心内执行次序边集** E_σ : 若在某个核心的执行序列 σ_k 中, 子图 s 紧邻排定在子图 t 之前执行, 则引入一条有向调度时序边 $(s, t) \in E_\sigma$ 。

将两类有向边与子图顶点集 $\mathcal{S}$ 合并, 构成加入同一核心顺序后的子图依赖图:

$$D^+(P) = (\mathcal{S}, E_{\mathcal{S}} \cup E_\sigma). \quad (3-1)$$

若该有向图中存在有向环, 则说明子图之间的数据流动与核心内的排队规则发生了冲突, 方案在官方前置派生与校验阶段 (如 `stub_multicore_cut_and_schedule.py` 第 220 行对商图环路的检查) 将因违反子图无环或执行顺序约束而被直接拒绝; 在问题一的独立任务硬件语义下, 此类冲突将直接表现为不同核心任务之间的循环等待。在图划分与多处理器调度领域, 将具有先后依赖的计算划分并保持商图 (quotient graph) 无环以及核心间负载平衡, 是受到广泛关注的课题 [2]; 然而, 通用无环图划分工具通常针对静态割权或通信量建模, 不能直接推导本题包含复杂硬件延迟与动态访存仲裁下的 Makespan 保证。

3.1.3 目标函数与可行域

在方案满足拓扑无环性及格式规范的前提下, 官方评测程序针对具体问题场景 $q \in \{1, 2, 3\}$ 执行离散事件模拟推进 $\mathcal{E}_q(P)$, 生成方案在规定硬件环境下的性能评估向量 $(M_q(P), D_q(P), H_q(P))$, 其中 $M_q(P)$ 为总体任务执行时间 (Makespan, 单位为 cycle), $D_q(P)$ 为额外 DDR 数据搬运量 (单位为 Byte), $H_q(P)$ 为问题三特有的 Cache 字节命中率。

设 $\mathcal{F}_q(G, K)$ 为评测程序能够顺利推进并正常完成的所有合法方案集合, 多核心调度问题的数学优化模型表述为:

$$\min_{P \in \mathcal{F}_q(G, K)} M_q(P). \quad (3-2)$$

多核心调度方案的官方评价目标 (题面 §1.3–1.4) 是以最小化任务总体执行时间 (Makespan) $M_q(P)$ 为首要优化目标, 同时考察额外 DDR 数据搬运量 (官方以相对单核基线的额外数据搬运量 `extra_ddr_bytes` 统计); 对于问题三, 官方评测程序同时报告基于字节统计的 Cache 命中率 $H_3(P)$ 。字典序比较并非官方评价指标的法定公式, 而是本队求解器在面临多个候选方案时采用的内部择优与回退保护规则: 在问题一与问题二中, 算法在候选方案之间按主次目标进行比较 (其中问题一在候选调度器内部按 $(M_1(P), \text{scheduled_copy_bytes})$ 排序择优, 最终在官方口径下折算为相对单核基线的 `extra_ddr_bytes`; 问题二在离散模拟评估后按 $(M_2(P), D_2(P))$ 字典序改善择优); 在问题三中, 算法则严格以缩短 Makespan ($M_3(P) < U$) 作为是否采纳 Cache 专用重排方案的门槛准则。

3.2 求解框架与方法选择

3.2.1 结构化候选方案的构造与筛选

求解该多核调度问题的根本难点在于: 图论意义上的局部启发式指标 (如划分割边权重、计算量均匀度) 无法准确反映带有动态带宽争用与复杂同步等待的微架构执行过程; 然而, 若对搜索空间中的海量可能切分逐一运行完整的离散事件模拟评测, 其时间开销在工程上又是不可接受的。

为了在合理的时间内产出高质量方案, 本文在方法论层面对三个问题的求解策略进行了通用流程归纳:

1. **图拓扑特征分析**: 分析原始计算图的连通分支、长程关键路径以及张量复用聚集度, 提取适合多核心展开的天然结构单元;
2. **结构化候选方案构造**: 针对提取出的拓扑特征, 直接生成少量包含切分、核心分配与执行顺序的完整候选方案 $P = (\pi, a, \sigma)$;
3. **拓扑合法性与无环性校验**: 构建扩展依赖图 $D^+(P)$, 校验所有待调度算子的无遗漏覆盖以及子图任务执行时序的无环性;
4. **条件支持的下界评估与剪枝**: 在具备确定性下界推导的特定算法分支中 (如问题三中的流水线计算下界), 根据必要时序约束计算完成时间下界, 排除在 Makespan 指标上绝无可能超越当前最优解的候选方案;
5. **受限预算下的离散事件模拟选优**: 对通过筛选的少量优质候选方案, 调用离散事件评测程序获取模拟性能指标, 自适应输出 Makespan 最小的方案。

图 3-1 概括了上述五步求解流程的通用方法逻辑。需要明确的是, 上述流程属于对各题求解思想的抽象归纳, 在软件实现上, 三个问题对应三个独立且已固化的求解器程序, 其入口脚本、候选生成逻辑与在线评价工具均不相同 (问题一入口为 `src.q1.branch_refine`, 在线调用至多 10 次轻量评估器 E1; 问题二入口为 `adaptive_hyergap_guarded.py`, 在线调用至多 3 次原生评估器 E2; 问题三入口为 `src.q3.forest_solve`, 各阶段共享至多 3 次官方仿真器 E0)。贯穿全篇的核心设计准则体现为三点: **以结构化构造配合拓扑无环检查、以图特征约束有限候选数量, 以及在紧凑的在线评测预算内以仿真结果闭环选优**。具体基于标号偏序的无环性证明留待后文命题展开; 理论下界剪枝仅在具备严格证明条件的分支中启用, 不声称三道题目统一实施了硬性下界剪枝。

根据题面硬件执行语义的演变, 三类问题的具体构造策略呈现清晰的递进逻辑:

- **问题一 (子图隔离执行)**: 每个子图独立封装为一个硬件 Task, 子图间数据无法片上保留, 通信代价高昂。算法侧重于识别无数据依赖的并行支路, 通过分支重新分配均衡各核心的计算负载, 平衡并行度与 DDR 通信开销;
- **问题二 (同核子图合并)**: 同一核心上的全部子图合并为单一 Task, 允许片上私有缓存直接跨子图复用数据。算法基于超图划分理论, 将具有密集张量复用的算子归并至同一核心, 以减少跨核心边界的数据通信, 并在局部最小割过程中通过按流水线的工作量上限保护机制防止计算向单核过度倾斜; 该工作量保护并不构成消除 Step 2 缓存换入换出 (spill) 的容量证明, 方案的物理表现由后续评测综合决定;
- **问题三 (引入共享只读 Cache)**: 在问题二合并执行的基础上增加核外只读 L2 Cache。针对多核心读取共享数据的树状归约结构, 算法基于深度优先排序理论排定子树计算次序, 在非交错执行模型下降低限定模型内部峰值; 最终的 Cache 字节命中率与整图 Makespan 仍由官方离散事件模拟评测器综合评定。

3.2.2 相关研究与方法适用性

多处理器任务调度与深度学习编译优化领域的研究为本文提供了重要的理论参照, 但在问题设定与优化自由度上存在明确区别:

1. **深度学习编译优化与搜索空间控制**: 张洪滨等提出的 AutoConfig 机制 [3] 通过综合静态特征与动态运行开销来缩减自动调优搜索空间; 高伟等在 TVM 框架下 [4] 利用预定义规则与代价模型裁剪算子调度空间。上述工作体现了在编译优化中兼顾方案质量与搜索耗时的平衡理念。与端到端代码生成系统不同, 本赛题的提交自由度严格限定于官方接口约定的算子划分、核心分配与执行顺序三个维度, 不涉及算子内部循环变换或底层微内核代码生成;

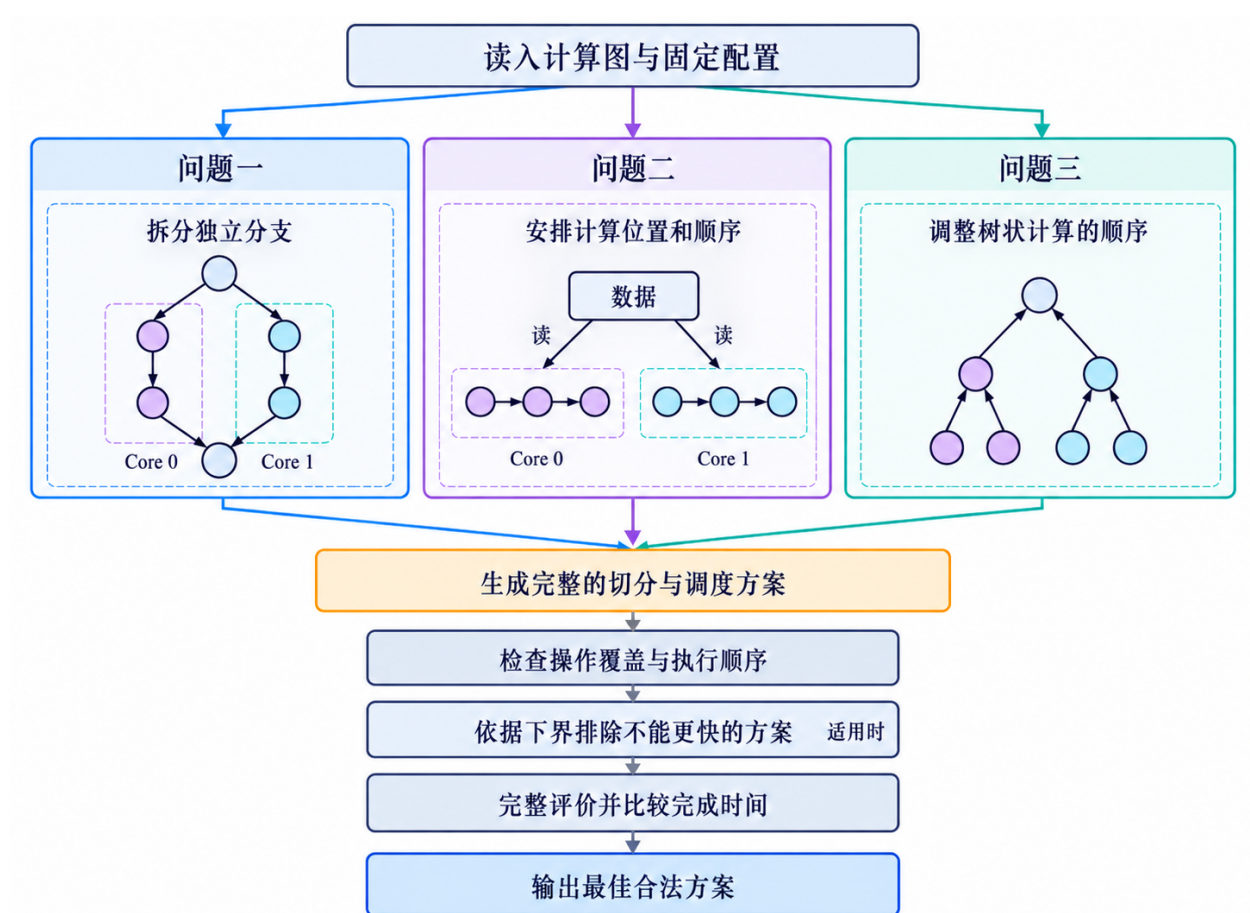


图 3-1 三问方案构造与选优流程

图注：三问共用的方案构造与比较步骤。各分支只在结构条件满足时采用，所形成的完整方案需检查依赖与容量，并通过执行评价比较完成时间。下界只在其证明条件成立时用于提前排除候选。

2. **经典多处理器与异构计算调度：**在异构 DAG 调度领域，HEFT 算法 [5] 通过静态计算与通信成本预估建立任务优先级与最早完成时间排程，是经典启发式基准。然而，HEFT 依赖静态确定的传输延迟模型，无法直接适应本题中多核心共享 DDR 时的动态总线争用、片上私有缓存容量限制以及复杂的事件驱动推进机制；
3. **理想化车间模型与理论边界：**在运筹调度领域，Drobouchevitch 与 Strusevich 针对两机重入车间模型 [6] 证明了特定工件流向下的多项式时间 $4/3$ 近似界。该类经典理论有助于理解反复进出执行资源的排队时序边界，但其理想化假设无法覆盖本题包含多类硬件执行流水线（Cube/Vector/MTE）、共享只读 Cache 及离散事件仲裁的实际执行环境，其理论近似比不能直接迁移至本题；
4. **神经网络处理器调度与存储约束：**针对 NPU 的调度研究（如 LATTICE[7]）表明，执行次序的调整与片上存储规划存在密切的相互影响。在问题设定上，本文的决策空间严格限定于算子划分、分核指派与执行次序三个维度，不能直接复用针对底层物理存储块的联合规划方法；各求解器中采用的流水线工作量上限或树模型启发式亦不能代替片上缓存的容量证明，方案最终的合法性与完成时间仍由本赛题的官方离散事件评测器统一核定。

3.3 可行性判定与完成时间下界

3.3.1 扩展依赖图的无环性判定

通过以下反例，我们可以说明为什么仅仅保证原始计算图无环以及每个子图内部无环，并不能确保多核心调度方案可安全执行。

假设原始计算图包含四个计算算子 a_1, a_2, b_1, b_2 ，它们之间仅存在两条数据依赖边 $a_1 \rightarrow b_1$ 与 $b_2 \rightarrow a_2$ 。显然，原始计算图是一个严格的有向无环图。若算法将算子划分为两个子图 $S_A = \{a_1, a_2\}$ 与 $S_B = \{b_1, b_2\}$ ，并分别分配至核心 0 与核心 1 执行。此时，子图依赖商图同时存在由 S_A 到 S_B 的依赖 ($a_1 \rightarrow b_1$) 以及由 S_B 到 S_A 的依赖 ($b_2 \rightarrow a_2$)。无论两个核心内部如何排定子图的执行次序，该方案都会因违反官方子图无环与执行顺序约束而在前置派生阶段被直接拒绝；在问题一的独立硬件任务执行语义下，核心 0 上的任务与核心 1 上的任务更将陷入相互等待对方完成的循环等待。不仅如此，即便子图间仅存在单向数据流动，若核心内部指定的调度序列引入了反向时序边，两类边叠加后同样会使扩展依赖图成环并被系统拦截。

因此，在对候选方案进行深入评价之前，必须形式化判定其扩展依赖图 $D^+(P)$ 的无环性。

命题 3.1 (无环性充分条件)。设 $P = (\pi, a, \sigma)$ 为一份多核心切分与调度方案，其对应的扩展依赖图为 $D^+(P) = (\mathcal{S}, E_S \cup E_\sigma)$ 。若存在一个标号函数 $r : \mathcal{S} \rightarrow \mathbb{Z}$ ，使得对于图中任意有向边 $(s, t) \in E_S \cup E_\sigma$ ，均满足

$$r(s) < r(t), \quad (3-3)$$

则该扩展依赖图 $D^+(P)$ 必然不存在有向环，方案中的子图任务能够排定出合法的拓扑执行序列。

证明. 采用反证法。假设 $D^+(P)$ 中存在一个由 m 个子图构成的有向环 $s_1 \rightarrow s_2 \rightarrow \cdots \rightarrow s_m \rightarrow s_1$ (其中 $m \geq 2$)。根据命题假设，环上的每一条有向边 (s_i, s_{i+1}) 均满足严格的大小偏序关系：

$$r(s_1) < r(s_2) < \cdots < r(s_m) < r(s_1). \quad (3-4)$$

由此直接导出 $r(s_1) < r(s_1)$ ，出现数学逻辑矛盾。因此，假设不成立， $D^+(P)$ 中必然不存在有向环。□

需要明确的是，标号函数 r 是用于证明执行顺序不存在循环互锁的拓扑排序判据，而非用于衡量调度性能的评分函数。通过有序块号与标号偏序证明商图无环性在无环 DAG 划分数学建模 [8] 以及带有无环约束的图划分算法 [9] 中有着经典应用。在有向图划分相关研究中，Moreira 等 [9] 探讨了沿拓扑序将连续节点块指派至同一分块以保障商图无环的构造方法；区别于特定拓扑序连续分块，本文命题 3.1 给出了适用于任意严格单调标号的一般无环性充分条件，用于判定多核调度方案不存在循环死锁。通过该判定仅证明了调度时序在依赖关系上的无环可行性；方案在实际推进中是否会出现片上缓存超限或评测异常，仍须通过后续的离散事件模拟进行完整验证。

3.3.2 完成时间下界与候选剪枝

为了在受限时间内高效筛选候选解，我们需要建立问题固有的理论时间下界。在异构流水线与多核心调度中，简单的贪心启发式排程容易引发多处理器调度异例（例如增加资源或减少局部耗时反而增大全局完成时间）[10]；因此需要结合物理必要不等式推导严格的完成时间下界。

固定输入图 G 、核心总数 K 以及问题编号 q 。若对于集合 $\mathcal{F}_q(G, K)$ 中的任意合法方案 P ，其 Makespan 均满足 $M_q(P) \geq L$ ，则称 L 为该问题的**完成时间下界 (lower bound on makespan)**，其单位为时钟周期 (cycle)。

我们通过考察所有合法方案都必须完成的固有计算工作量与依赖链长，推导一个全局基础下界：

1. **计算算子子集与周期定义：** 设 $V_{\text{comp}} \subseteq V$ 为具有确定执行时长的计算算子集合，它由绑定在矩阵流水线 PIPE_M (Cube 单元) 与向量流水线 PIPE_V (Vector 单元) 上的计算操作构成，其算子固有计算耗时严格定义为 $d_v = \max(1, \text{cycles}_v)$ (单位为 cycle)。集合 $V \setminus V_{\text{comp}}$ 中的数据搬运与内存访问操作耗时取决于底层总线仲裁、动态带宽争用与缓存状态，不具有静态固定常数周期；若输入图中缺乏某一特定流水线操作 (例如 $V_M = \emptyset$ 或 $V_V = \emptyset$)，则对应阶段的总工作量取为边界值 0 (即 $W_M = 0$ 或 $W_V = 0$)；
2. **异构计算负载下界：** Cube 密集型计算算子集合记为 $V_M \subseteq V_{\text{comp}}$ ，Vector 密集型计算算子集合记为 $V_V \subseteq V_{\text{comp}}$ 。其总计算量分别为 $W_M = \sum_{v \in V_M} d_v$ 与 $W_V = \sum_{v \in V_V} d_v$ (cycle)。由于全系统包含 K 个计算核心，且每个核心均配备物理独立的 PIPE_M 与 PIPE_V 硬件执行流水线，受算子不可切分及局部依赖约束，零通信理想调度的最优执行时长并不必然等于工作量除以核心数；但任何合法方案完成各流水线计算所需的时间绝不可能低于工作量均分下界，即 $\lceil W_M/K \rceil$ 与 $\lceil W_V/K \rceil$ 构成了矩阵与向量计算的必要下界。由于两类计算流水线采用物理分离的执行单元，其计算负载不能简单相加除以 K ；
3. **关键路径依赖下界：** 在由全体计算算子 V_{comp} 构成的依赖关系中，若算子 u 的输出张量直接或通过中间数据搬运操作传递给算子 v 读取，则存在从 u 到 v 的计算依赖路径。在离散事件动态系统理论中，对于任务时长与依赖关系固定的同步系统，各操作在其前驱算子均完成的最晚时刻启动，沿依赖路径累加各操作时长可给出完成时间的必要下界 [11, 第 11–13、109–110 页]；需要指出的是，该经典下界背景建立在固定时延模型之上，并不涵盖多核共享 DDR 动态争用或 FIFO 缓存逐出等状态，本文亦不将全流程仿真简化为固定线性系统。在赛题框架下，将原始图中跨核心数据搬运链 (COPY 路径) 投影或折叠为计算算子间的依赖弧，仅在满足附录 B.1 经官方评测代码验证的硬件同步与依赖推导语义下成立，不能无条件断言任意复杂搬运链均可简单等价缩点；定义从源点算子到汇点算子沿任意经核验的计算依赖路径累加各算子计算时长 d_v 的最大值为**关键路径长度** L_{cp} (cycle)，该累加严格计入路径终点算子的计算时长 d_v 。由于同一依赖链上的算子在逻辑上必须依次串行执行，任何合法方案的全局执行时间必然不低于 L_{cp} 。

综合上述约束，针对全局所有合法方案成立的理论基础下界为：

$$L_{\text{basic}} = \max \left\{ L_{\text{cp}}, \left\lceil \frac{W_M}{K} \right\rceil, \left\lceil \frac{W_V}{K} \right\rceil, \max_{v \in V_{\text{comp}}} d_v \right\} \leq M_q(P). \quad (3-5)$$

当特定流水线操作集合为空时，其对应分量取为 0。

除了适用于全部方案的全局下界外，针对某一具体候选方案 $P = (\pi, a, \sigma)$ ，若考虑其特定的子图分配、各核心实际承担的算子工作量峰值以及跨核心传输所必需的最低延迟，可以推导出针对该方案的特定下界 $L(P) \leq M_q(P)$ 。

基于特定下界 $L(P)$ ，可建立针对主目标的**理论剪枝判据**：假设在求解过程中算法已获得一份经过模拟评测确认的合法方案 P_0 ，其 Makespan 记为当前性能上界 $U = M_q(P_0)$ 。

若后续搜索旨在寻找 **Makespan** 严格更短的解（即满足 $M_q(P) < U$ ），由于 $M_q(P) \geq L(P)$ ，一旦新候选方案 P 的特定下界满足

$$L(P) \geq U, \quad (3-6)$$

则必然有 $M_q(P) \geq U$ ，该候选在完成时间维度绝无可能严格超越当前已知最优解 P_0 。需要指出的是，该判定仅排除在 **Makespan** 维度不可能严格改善的方案，并不等同于过滤了所有劣质解；若在相同周期时允许通过次要目标（减少额外搬运量 $D_q(P)$ ）选优，则在 $L(P) = U$ 的边界情形下，下界无法直接排除该候选产生更小搬运量的可能性。在实际算法实现中，全局理论下界并未作为三个问题统一执行的硬性搜索剪枝器，而是在具备确定下界推导的特定算法分支（如问题三中的流水线计算下界）中作为辅助判据；各题主要通过轻量级结构校验与有限高质量候选策略来规避无效搜索。

若 L 为全局下界， $\text{OPT} = \min_{P \in \mathcal{F}_q(G, K)} M_q(P)$ 为当前用例在给定核心数下的未知数学最优值，而 U 为算法所得最佳方案的实际 **Makespan**，三者在线纲上均为 cycle 且满足 $L \leq \text{OPT} \leq U$ 。当 $L > 0$ 时，同线纲相除得到无量纲的最优性相对间隙上界：

$$\frac{U - \text{OPT}}{\text{OPT}} \leq \frac{U - L}{L} = \frac{U}{L} - 1. \quad (3-7)$$

比值 U/L 在数学上给出了算法当前方案偏离未知理论最优解的保守相对误差上限 $\frac{U}{L} - 1$ 。比值越接近 1，表明理论上可证明的最大相对误差越小；需要强调的是，该比值仅反映相对于特定松弛下界的可证明误差上限，并不等同于当前方案距离真实未知最优解的精确绝对距离。

3.4 求解耗时与计算复杂度

求解程序的运行效率通过墙钟时间进行度量，它与底层离散事件模拟时钟周期（**Makespan**）具有本质不同的物理内涵。

求解程序的端到端运行时间 T_{solve} （以秒为单位）由以下五个执行阶段构成：

$$T_{\text{solve}} = T_{\text{parse}} + T_{\text{construct}} + T_{\text{check}} + \sum_{j=1}^b T_j + T_{\text{IO}}, \quad (3-8)$$

其中：

- T_{parse} 为输入计算图 JSON 文件的解析与数据结构初始化时间；
- $T_{\text{construct}}$ 为针对特定问题场景生成有限候选方案的算法构造时间；
- T_{check} 为算子覆盖合法性与扩展依赖图 $D^+(P)$ 无环性判定时间；
- b 为算法在求解当前用例期间实际调用的在线离散事件评测次数， T_j 为第 j 次在线评测的运行耗时；
- T_{IO} 为将最终优选方案持久化写入标准输出 JSON 文件的磁盘交互时间。

在计算复杂度层面：

1. **拓扑解析与无环校验**：设待调度算子数为 $|V|$ ，直接与间接数据依赖边数为 $|E_V|$ ，生成的子图数量为 $|\mathcal{S}|$ 。在扩展依赖图 $D^+(P)$ 边集已显式构建的前提下，基于 Kahn 算法 [12] 的有向无环性判定可在 $O(|V| + |E_V| + |\mathcal{S}|)$ 的线性时间内完成。需要指出的是，该线性

界仅适用于显式图的拓扑排序本身，并不包含此前跨边界 COPY 路径的提取投影、确定性字典序排序以及各候选方案的具体构造耗时；

2. **结构候选构造与软预算约束：**结构化构造主要包含连通分量遍历、前沿划分与支路重分配检查。各算法针对深层结构分析设定了工程规模与时间上限（例如问题一中限制任务总数不超过 320，并引入 2.0 秒的结构分析软预算检测点，若超时则主动退出深层探索并安全回退）。此类工程保护机制旨在规避极端复杂拓扑下的退化搜索，并不构成整体端到端求解时间的渐近复杂度保证；
3. **离散事件模拟评测开销：**单次离散事件评测耗时 T_j 取决于评测内核在推进离散事件时的队列操作与硬件模拟规模。本文各求解器将单用例的在线评测调用次数 b 限制在固定常数上限内（问题一至多 10 次，问题二与问题三至多 3 次），避免了评测次数随图规模无限制累加。结合前置结构校验与分支守卫，实测端到端求解耗时（具体数值见第 7 章）处于工程实用范围之内；赛题第 5 页脚注 1 所提的 5~10 分钟属于避免暴力迭代的效率建议，并非算法运行耗时的下限指标。

4 问题一：独立任务执行下的多核调度

4.1 问题分析与时序约束

在问题一的系统设定中，每个非空子图 $s \in \mathcal{S}$ 均被底层编译系统独立封装为一个物理硬件任务（Task）。由于子图之间在硬件执行层面上严格隔离，芯片私有缓存无法实现跨 Task 的数据保留。无论两个具有数据依赖关系的 Task 最终被指派至同一个计算核心还是不同的计算核心，它们之间传递的中间数据张量均必须通过外部主存（DDR）进行持久化中转，即由前驱 Task 将生成张量执行 COPY_OUT 写出至 DDR，再由后继 Task 执行 COPY_IN 从 DDR 读回片上缓存。

设 Task s 在指派核心 $a(s)$ 上的开始执行时刻与完成时刻分别为 A_s 和 F_s （单位为 cycle）。记 $\text{prev}(s)$ 为排定在同一核心上紧邻 s 之前执行的 Task， $\text{pred}(s)$ 为在数据依赖关系上先于 s 执行的全部前驱 Task 集合。底层离散事件评测系统对硬件调度时序施加了如下物理约束：

$$A_s \geq F_{\text{prev}(s)} + \Delta_{\text{local}}, \quad \Delta_{\text{local}} = 100 \text{ cycles}, \quad (4-1)$$

$$A_s \geq F_t + \Delta_{\text{sync}}, \quad \forall t \in \text{pred}(s) \text{ 且 } a(t) \neq a(s), \quad \Delta_{\text{sync}} = 1000 \text{ cycles}. \quad (4-2)$$

其中， $\Delta_{\text{local}} = 100 \text{ cycles}$ 为同一核心上相邻任务间的上下文切换开销； $\Delta_{\text{sync}} = 1000 \text{ cycles}$ 为跨核心依赖同步屏障引入的硬件等待时延。若某项约束中的前驱对象不存在，则对应不等式不生效。

上述时序约束揭示了多核心切分与调度决策中的基本权衡关系（trade-off）：

1. **并行收益与排队削减：**将互无计算依赖的算子划分至不同的子图并分配给多个计算核心，能够让无依赖的算子在物理分离的流水线（PIPE_M 与 PIPE_V）上并发推进。虽然这并不改变由原始数据依赖所决定的固有关键路径长度，但能够缩短由于单核执行资源有限而被强制串行化的排队等待时间；
2. **通信开销与同步代价：**若子图切分引入了跨子图的数据流动边界，中间数据张量必须通过 DDR 进行读写传递，从而产生额外的数据搬运开销与总线争用；当数据依赖跨越不